Universidade Federal do Maranhão
Centro de Ciências Exatas e Tecnologia — Departamento de Engenharia da Computação
Disciplina: Sistemas Distribuídos — Docente: Prof. Dr. Luiz Henrique Neves Rodrigues

Relatório Técnico de Engenharia de Middleware

Consolidação e Migração para Arquitetura Message-Oriented Middleware com Apache Kafka: Particionamento Determinístico, Consumer Groups e Resiliência Adaptativa

Discentes: Eduardo dos Santos Oliveira, Fernando da Silva Costa, Herick Vinicius Pinheiro Da Conceição, Samara Santos Viegas e Vyktor Gabryel Costa de Souza

---

1 Contextualização e Decisão Arquitetural

O processamento distribuído de telemetria hospitalar impõe requisitos de missão crítica: alta vazão (*throughput*) para dezenas de milhares de medições por segundo, garantia estrita de ordenação temporal dos sinais vitais por paciente, tolerância a falhas em cascata e replicação paralela sem degradação do emissor.

Após a análise comparativa entre diferentes paradigmas de comunicação distribuída, a equipe consolidou a substituição de todos os middlewares legados (RabbitMQ AMQP e barramentos híbridos) em favor do **Apache Kafka** como a **espinha dorsal única e definitiva** de mensageria (*Message-Oriented Middleware - MOM*):

1. **Ordenação Temporal Garantida por Chave de Partição:** No RabbitMQ convencional, filas FIFO distribuídas perdem a garantia de ordenação estrita quando múltiplos consumidores competem na mesma fila. No Apache Kafka, ao particionar o tópico pela chave `paciente_id`, todas as medições de um mesmo paciente são roteadas determinística e sequencialmente para a mesma partição física, eliminando condições de corrida clínicas.

2. **Modo KRaft (*Kafka Raft Metadata Mode*):** O broker opera no padrão moderno do Apache Kafka sem qualquer dependência legada de clusters ZooKeeper. O consenso do cluster é mantido nativamente pelo algoritmo Raft interno, reduzindo o *overhead* operacional e simplificando o empacotamento em containers.

3. **Replicação Fanout via Consumer Groups Desacoplados:** Em vez de criar exchanges duplicadas ou intermediários pesados, a difusão simultânea dos dados para os 4 servidores backend é realizada através de *Consumer Groups* independentes (`consumidor_servidor_1` a `consumidor_servidor_4`). Cada servidor lê do mesmo tópico com offset independente e velocidade própria.

Fundamento de Engenharia:

Log de Commit Distribuído Imutável (Jay Kreps) Diferente de sistemas de filas tradicionais onde mensagens são apagadas após o consumo (*destructive read*), o Apache Kafka modela os dados como um log sequencial somente de escrita (*append-only commit log*). As mensagens persistem no disco e são particionadas, permitindo releitura de dados históricos para auditoria médica e consistência eventual sem sobrecarga no broker.

2 Arquitetura e Implementação do Middleware Kafka

A camada de middleware foi estruturada de forma modular no pacote `middleware/`, contendo os componentes essenciais de produção, consumo, resiliência e configuração:

Padrão Arquitetural Kafka:

Componentes do Middleware Kafka (`middleware/`)

- `config.py`: Configuração centralizada com suporte a injeção via variáveis de ambiente (`KAFKA_BOOTSTRAP_SERVERS`, `KAFKA_TOPIC`), especificação de 4 partições, fator de replicação, parâmetros de buffer (`batch.size = 16KB`, `linger.ms = 5`) e mapeamento dos 4 consumer groups de replicação.
- `produtor.py`: Classe `ProdutorKafka` com serialização JSON, particionamento por hash da chave `paciente_id`, compressão nativa (*gzip*), semântica idempotente (`enable.idempotence = True`, *exactly-once semantics*) e controle de vazão por *Token Bucket Rate Limiting*.
- `consumidor.py`: Classe `ConsumidorKafka` com commit manual de offsets (*at-least-once delivery*), máquina de estados de *Circuit Breaker* por nó e isolamento de erros em *Dead Letter Topic (DLT)*.

2.1 Particionamento e Paralelismo de Escrita

O tópico mestre `telemetria_hospitalar` é configurado com 4 partições. A atribuição de partição para cada registro de telemetria é calculada segundo a função de dispersão determinística:

$$P_i = \text{murmur2}(\text{paciente_id}) \pmod{N_{\text{partições}}} \quad (1)$$

Essa distribuição garante que a telemetria do mesmo paciente seja processada em ordem estrita de chegada, ao mesmo tempo em que distribui a carga uniformemente entre os nós.

2.2 Replicação Fanout nos 4 Servidores Backend

Para garantir consistência e tolerância a falhas (Marco 3), cada nó do cluster hospitalar possui um banco de dados independente (`server1.db` a `server4.db`). O consumidor Kafka opera no modo `-multi`, instanciando 4 threads concorrentes:

Consumer Group	**Destino**	Papel na Arquitetura
`consumidor_servidor_1`	`http://localhost:8001`	Nó Ativo Principal 1 (`server1.db`)
`consumidor_servidor_2`	`http://localhost:8002`	Nó Ativo Principal 2 (`server2.db`)
`consumidor_servidor_3`	`http://localhost:8003`	Nó Ativo Principal 3 (`server3.db`)
`consumidor_servidor_4`	`http://localhost:8004`	Nó de Reserva Aquecida (`server4.db` - Warm Standby)

Tabela 1: Mapeamento de Consumer Groups do Kafka para a replicação ativa nos shards de banco de dados.

3 Tolerância a Falhas e Mecanismos de Proteção

3.1 Circuit Breaker Integrado

Para evitar que a indisponibilidade transitória de um dos servidores sature o consumidor com retentativas infinitas, cada consumidor possui um **Circuit Breaker** com três estados:

- **CLOSED (Fechado):** Operação nominal. Requisições POST são despachadas normalmente ao servidor backend.
- **OPEN (Aberto):** Após 5 falhas consecutivas de conexão (ou timeout HTTP), o circuito desarma imediatamente por 10 segundos, retendo o offset da mensagem sem commit e evitando sobrecarga.
- **HALF-OPEN (Semi-aberto):** Transcorrido o período de resfriamento, o circuito permite requisições de teste. Se duas consecutivas obtiverem sucesso (HTTP 200), o circuito retorna a **CLOSED**.

3.2 Dead Letter Topic (DLT) com Exponential Backoff

Mensagens com dados corrompidos ou que esgotem o limite de tentativas ($N = 3$) são encaminhadas para o tópico de quarentena `telemetria_hospitalar_dlt`:

$$t_{\text{backoff}} = \min(t_{\text{máx}}, t_{\text{base}} \times 2^{\text{tentativa}}) \tag{2}$$

Esse mecanismo assegura que dados inválidos não bloqueiem a leitura contínua das mensagens sadias dos demais leitos hospitalares.

3.3 Failover Crítico On-Demand e Auto-Scaling Dinâmico (Servidor 04)

Durante experimentos de *Engenharia do Caos* na interface gráfica, simulou-se o cenário extremo em que os 3 servidores principais (`server-01`, `server-02` e `server-03`) foram derrubados simultaneamente, restando apenas o nó de reserva (`server-04`) em *Warm Standby*.

Para garantir alta disponibilidade imediata e transparência operacional, foram implementadas as seguintes soluções arquiteturais:

1. **Detecção Instantânea de Queda de Processo:** O *Health Checker* do balanceador foi otimizado para que erros de conexão recusada (`httpx.ConnectError`) marquem imediatamente o nó como `UNHEALTHY` logo na primeira sondagem, eliminando a latência de espera de 9 segundos (três ciclos convencionais de timeout).

2. **Ativação On-Demand em Roteamento de Requisição:** Se uma requisição HTTP (`POST /dados` ou `GET /dados`) chega ao balanceador e todos os nós principais falharam, o balanceador invoca imediatamente `ativar_reserva()` sob demanda, roteia a requisição diretamente para o Servidor 04 (:8004) e retorna sucesso (`HTTP 200 OK` com `X-Failover: true`), sem devolver erros `502 Bad Gateway`.

3. **Notificação Reativa e Sincronização Instantânea com o Monitor:** O balanceador despacha uma notificação assíncrona para o endpoint `POST /api/evento-escala-externo` do monitor de visualização. O monitor valida o processo do nó 4 e emite imediatamente os eventos SSE `auto_scale_event` e `escala_alterada` para o navegador.

4. **Transição Visual Automática na Interface Web:** No frontend, o nó 4 transita instantaneamente de `STANDBY` para `ONLINE / ATIVO (25%)`, o botão comuta para *Desativar Reserva (Scale-In)*, o SVG da arquitetura ilumina a rota física do Servidor 04 e a rotação do Round Robin passa a incluir o quarto nó.

4 Resultados dos Testes Automatizados

A consolidação da arquitetura Kafka e dos mecanismos de escalabilidade foi acompanhada da criação e expansão de baterias de testes unitários e de integração em `tests/`:

- Validação de carregamento e sobrescrita de configurações do Kafka via variáveis de ambiente.
- Algoritmo Token Bucket para controle de vazão (admissão de rajadas e estrangulamento).
- Transições completas de estado do Circuit Breaker (`CLOSED` → `OPEN` → `HALF-OPEN` → `CLOSED`).
- Conversão e sanitização do modelo canônico de dados hospitalares com Pydantic.
- Failover de emergência do auto-scaler quando todos os nós caem e ativação sob demanda.
- Notificação reativa e endpoints de eventos de escala externa no monitor de visualização.

O resultado consolidado da execução da suíte de testes com `pytest` atesta a estabilidade rigorosa do sistema:

Módulo de Teste	Testes Executados	Status e Cobertura
`tests/test_kafka_middleware.py`	11	**100% Aprovados** (10 unitários + 1 integração)
`tests/test_marco1.py`	16	**100% Aprovados** (Dataset, Pydantic, SQLite)
`tests/test_marco2.py`	9	**100% Aprovados** (Balanceador, Round Robin)
`tests/test_marco3.py`	7	**100% Aprovados** (Replicação Fanout, Escalabilidade)
`tests/test_escalabilidade.py`	35	**100% Aprovados** (Algoritmos e Auto-scaling)
`tests/test_visualizacao.py`	20	**100% Aprovados** (SSE, SSE Web, Métricas Kafka, Escala Externa)
Total Consolidado	**98 Testes**	**100% Aprovados (Zero Falhas)**

Tabela 2: Resultados da suíte automatizada de testes após a migração para o Apache Kafka e aprimoramento do failover.

5 Conclusão

A consolidação do **Apache Kafka** como middleware exclusivo, combinada com o **Balanceador Enterprise com Failover On-Demand e Auto-Scaling Dinâmico**, conferiu ao Sistema Distribuído Hospitalar um patamar industrial de estabilidade, desempenho e transparência. A combinação de particionamento estrito por paciente, semântica idempotente de envio, replicação paralela via Consumer Groups, tolerância a falhas via Circuit Breakers e recuperação imediata do Servidor Reserva garante que o sistema atenda integralmente a todos os requisitos acadêmicos e funcionais da disciplina de Sistemas Distribuídos da UFMA.